

Teacher's Guide

Teaching Docker with Spring Boot, Next.js and MySQL

5 sessions · about 90 minutes each · timings, talking points, live demos, answer key

1. Course at a glance

The idea: students get a working app with **no Docker setup** (the student - starter / folder) and write every Docker file themselves. Each session starts with a real problem that the previous one left behind, and the new tool solves it.

#	Topic	Students leave able to...	Problem that leads to the next session
1	Images and containers	run, inspect, stop and remove containers; start MySQL with env vars	"How do I put <i>my own</i> app in a container?"
2	Dockerize Spring Boot	write a single-stage Dockerfile; build, tag and run it; connect to MySQL via a network	The image is 1.1 GB, and every code change re-downloads all dependencies
3	Multi-stage and caching	split build and runtime; order layers for caching; use .dockerignore; run as non-root	"Now the frontend, and how do they talk to each other?"
4	Frontend and networking	Dockerize Next.js; explain service-name DNS; avoid the localhost trap	Too many manual commands, run in the right order
5	Docker Compose	write a compose file with volumes, healthchecks, depends_on and networks	Final assignment

Materials

What	Where	Used in
Student starter (app code + TODO files)	student-starter/	Every session: give it out as a zip
Answer key (complete project)	DockerBackend/, DockerFe/, docker-compose.yml	You only. Show parts after each session
Docker Cheat Sheet	Docker-Cheat-Sheet.pdf	From session 1, kept open
Line-by-line explanation	Dockerfiles-and-Compose-Explained.pdf	Hand out after sessions 3, 4 and 5
Setup & run guide	Docker-Setup-Guide.pdf	Session 5 and homework
Full lesson text	README.md (root)	Reference for you and advanced students

Watch out: Give out **only** student - starter /. The root folder contains the full answers.

2. Preparation checklist

One week before

Done	Task
[]	Students install Docker Desktop and run <code>docker run --rm hello-world</code> . Collect screenshots, so nobody shows up with a broken install.
[]	Students pre-pull the base images at home (about 1 GB). Without this, classroom Wi-Fi can cost you the whole first hour:

Done	Task
	<code>docker pull mysql:8.4 maven:3.9-eclipse-temurin-21 eclipse-temurin:21-jre node:20-alpine</code>
[]	Windows students: Docker Desktop with the WSL 2 backend. Have them use Git Bash or WSL for the \-continued commands.
[]	Zip student-starter/ and share it.

On the day

Done	Task
[]	Your machine: build the answer key once with <code>docker compose up --build</code> so all layers are cached and the demos are instant.
[]	Free ports 3000 and 8080: <code>lsof -i :3000</code> and <code>lsof -i :8080</code> . Stop any dev servers.
[]	Terminal font at 20pt or more. Use a light theme on the projector.
[]	Offline plan: <code>docker save mysql:8.4 node:20-alpine ... -o images.tar</code> on a USB stick. Students run <code>docker load -i images.tar</code> .

Session 1: Images and containers

Goal: students understand image vs container and can manage containers from the command line.

Timing

Min	Activity
0-10	Hook: "It works on my machine." Ask who has had a project fail on someone else's laptop. Docker ships the machine along with the app.
10-25	Image = class, container = object. One image can run many containers. Draw it on the board.
25-40	Live demo (below).
40-75	Students start MySQL, exec into it, run SQL, then stop / start / remove it.
75-90	Recap quiz (questions 1-3 in section 5). Preview: "Next time, we put <i>our</i> app in a container."

Talking points

- A container is a **process** with its own isolated filesystem and network. It is not a virtual machine: there's no separate OS kernel, so it starts in under a second.
- Images are built in **layers** and are read-only. A container adds a thin writable layer on top. Remove the container and that layer is gone.
- `-e` configures a container from the outside. The same image works in dev and in production with different settings.
- `docker ps` shows running containers and `docker ps -a` shows all of them. Stopped containers still exist until you `rm` them.

Live demo

```
docker run hello-world                # what happened? (pull, create, run, exit)
docker run -it alpine sh              # a Linux shell in under a second
  / # cat /etc/os-release ; ls ; exit
docker ps -a                          # the exited containers are still there
docker rm $(docker ps -aq -f status=exited)

docker run -d --name db -e MYSQL_ROOT_PASSWORD=root_password \
  -e MYSQL_DATABASE=messages_db -e MYSQL_USER=app -e MYSQL_PASSWORD=app_password mysql:8.4
docker logs -f db                     # "ready for connections"
docker exec -it db mysql -u app -papp_password messages_db
```

Note: Ask: "Where is MySQL installed?" (Answer: nowhere on my laptop. It's only inside the container.)

Student task

Session 1 in student-starter/README.md: start MySQL, run `SHOW DATABASES;`, and practise `ps` / `stop` / `start` / `rm`.

Checkpoint (everyone must pass before moving on)

- [] Everyone can open a MySQL prompt inside their container.
- [] Everyone can explain the difference between `docker stop` and `docker rm`.

Errors you'll see in this session

Student sees	Cause and fix
Cannot connect to the Docker daemon	Docker Desktop isn't running. Start it and wait for the whale icon.
Conflict. The container name "/db" is already in use	A container with that name still exists (maybe stopped). Run <code>docker rm -f db</code> .
ERROR 2002 (HY000): Can't connect...	MySQL is still starting. Wait for "ready for connections" in <code>docker logs -f db</code> .

Note: Answer key: no files this session.

Session 2: Dockerize Spring Boot (single stage)

Goal: students write their first Dockerfile and run their own app in a container connected to MySQL.

Timing

Min	Activity
0-15	Walk through a Dockerfile instruction by instruction: FROM, WORKDIR, COPY, RUN, EXPOSE, ENTRYPOINT (cheat-sheet table).
15-25	Explain the build context : <code>docker build -t name ./DockerBackend</code> . Only files in that folder can be copied.
25-60	Students complete Part A of DockerBackend/Dockerfile, build it, and record the image size.
60-80	Create a network, run db + backend, then <code>curl</code> the API.

Min	Activity
80-90	Ask: "Why does DB_HOST=db work?" Leave the image size (1.1 GB) on the board as a cliffhanger.

Talking points

- Each instruction is one step of the build. RUN executes **while building**, and ENTRYPOINT/CMD runs **when the container starts**.
- EXPOSE is only documentation. -p 8080:8080 is what actually publishes the port. Write **HOST:CONTAINER** on the board.
- Containers on the same **user-defined network** find each other by **name**. Docker runs a small DNS server for that network.
- application.yml reads \${DB_HOST:localhost}, so the same jar works on a laptop and in Docker. Configuration comes from outside.

Live demo

```
docker build -t docker-backend ./DockerBackend
docker images docker-backend          # ~1.1 GB: remember this number!

docker network create lesson-net
docker run -d --name db --network lesson-net -e MYSQL_ROOT_PASSWORD=root_password \
  -e MYSQL_DATABASE=messages_db -e MYSQL_USER=app -e MYSQL_PASSWORD=app_password mysql:8.4
docker run -d --name backend --network lesson-net -p 8080:8080 \
  -e DB_HOST=db docker-backend
curl http://localhost:8080/api/health
curl -X POST localhost:8080/api/messages \
  -H "Content-Type: application/json" -d '{"text":"hi"}'
curl http://localhost:8080/api/messages
```

Note: Show the localhost mistake on purpose: run a second backend with -e DB_HOST=localhost and -p 8081:8080, then read docker logs. It fails with "Communications link failure". Inside a container, localhost is the container itself.

Student task

Part A of DockerBackend/Dockerfile (TODO 1-6). The reference solution is in the answer key box at the end of this session.

Checkpoint (everyone must pass before moving on)

- [] The /api/health endpoint answers {"status": "UP"}.
- [] Everyone has written down the image size (~1.1 GB).

Errors you'll see in this session

Student sees	Cause and fix
invalid tag ... repository name must be lowercase	Image names must be lowercase: docker-backend, not Docker-Backend.
Unable to access jarfile	Wrong path in ENTRYPOINT. The jar is target/DockerBackend-0.0.1-SNAPSHOT.jar.
COPY failed: ... not found	Wrong build context. Run the command from the project root with ./DockerBackend as the last argument.
Communications link failure	The backend can't reach MySQL. Check that DB_HOST=db, that both containers are on lesson-net, and that MySQL has finished starting.
port is already allocated	Something else is on 8080. Use -p 8081:8080 or stop the other program.

Note: Answer key: Part A (single stage) is: FROM maven:3.9-eclipse-temurin-21 /WORKDIR /app /COPY . . /RUN mvn -B package -DskipTests /EXPOSE 8080 /ENTRYPOINT ["java", "-jar", "target/DockerBackend-0.0.1-SNAPSHOT.jar"]. Tested: it builds to 1.11 GB.

Session 3: Multi-stage builds and layer caching

Goal: students shrink the image, make rebuilds fast, and run the app as a non-root user.

Timing

Min	Activity
0-15	Experiment first: change one string in ApiController.java and rebuild the Part A image. Everyone times it. All dependencies download again. Why?
15-30	Draw the layers as a stack. One changed layer invalidates everything above it. COPY . . early means any change rebuilds everything.
30-40	Multi-stage: the build tools are needed to make the jar, not to run it. Only the last stage ships.
40-75	Students write Part B plus .dockerignore, then repeat the timing experiment.
75-90	Compare sizes on the board (1.11 GB vs ~580 MB). Introduce USER and why to avoid running as root. Hand out the explainer PDF, section 2.

Talking points

- **Order matters:** put things that change rarely (pom.xml, dependencies) first and things that change often (src) last.
- COPY --from=build copies just the jar from the build stage. Maven, the JDK, the sources and ~/.m2 are all left behind.
- .dockerignore keeps target/, .git and secrets out of the build context. That gives faster builds and no leaks.
- Run as **non-root**: if the app is hacked, the attacker is not root inside the container.

Live demo

```
# before: Part A image, after a one-line code change
time docker build -t docker-backend ./DockerBackend      # minutes: all deps download

# after: Part B (multi-stage), after the same change
time docker build -t docker-backend ./DockerBackend      # seconds
# => CACHED [build 3/6] RUN mvn -B dependency:go-offline    <- point at this line

docker images docker-backend                             # ~580 MB (was 1.11 GB)
docker run --rm --entrypoint whoami docker-backend        # spring (not root)
```

Note: Then change `pom.xml` (just the description) and rebuild. It's slow again. Ask the class why.

Student task

Part B of `DockerBackend/Dockerfile` (TODO 7-14) and `DockerBackend/.dockerignore`.

Checkpoint (everyone must pass before moving on)

- [] Their build output shows CACHED for the dependency step after a code change.
- [] Their image is roughly half the size of Part A.
- [] `whoami` inside the container prints `spring`.

Errors you'll see in this session

Student sees	Cause and fix
<code>COPY --from=build ... not found</code>	The first stage isn't named: add <code>AS build</code> to the first FROM.
Dependencies still download on every build	<code>COPY src</code> (or <code>COPY . .</code>) comes before <code>dependency:go-offline</code> . Copy only <code>pom.xml</code> first.
<code>groupadd: command not found</code>	They used an Alpine-based JRE image. <code>eclipse-temurin:21-jre</code> is Ubuntu-based, so <code>groupadd</code> works.
Image still ~1 GB	They forgot the second FROM, or copied everything instead of just the jar.

Note: Answer key: `DockerBackend/Dockerfile` and `DockerBackend/.dockerignore` in the root folder.

Session 4: The frontend and Docker networking

Goal: students Dockerize Next.js and can explain exactly how a browser request reaches the backend.

Timing

Min	Activity
0-15	Show <code>next.config.ts</code> : output: <code>"standalone"</code> and the <code>/api</code> rewrite. The browser only talks to the frontend.
15-50	Students write the 3-stage <code>DockerFe/Dockerfile</code> and <code>.dockerignore</code> , then build and run it on <code>lesson-net</code> .
50-65	The localhost bug (demo below). Let them see it fail before you explain it.

Min	Activity
65-80	Draw the request path: browser -> localhost:3000 -> frontend container -> backend:8080 -> db:3306. Mark which parts are inside the Docker network.
80-90	Count the commands needed to run everything (about 7). "There must be a better way..."

Talking points

- The browser runs on **your laptop**, outside Docker, so it can never resolve backend. The Next.js server runs inside the network, so it can. That is why the rewrite exists, and a bonus is that no CORS setup is needed.
- Next.js calculates rewrites at **build time**, so BACKEND_URL must be a build ARG. Setting it only at runtime does nothing.
- HOSTNAME=0.0.0.0: the server must listen on all interfaces. Bound to localhost, nothing from outside the container could reach it.
- Standalone output plus copying only .next/standalone and .next/static gives an image of ~290 MB instead of shipping all of node_modules.

Live demo

```
docker build -t docker-frontend ./DockerFe
docker run -d --name frontend --network lesson-net -p 3000:3000 docker-frontend
# open http://localhost:3000 -> Backend status: UP

# The localhost bug:
docker build -t fe-broken --build-arg BACKEND_URL=http://localhost:8080 ./DockerFe
docker run -d --name fe-broken --network lesson-net -p 3002:3000 fe-broken
# open http://localhost:3002 -> Backend status: DOWN. Why?!
docker rm -f fe-broken
```

Note: Ask: "The backend IS on localhost:8080. I can curl it from my laptop. So why does this fail?" (Answer: the request is made by the frontend *container*, and its localhost is itself.)

Student task

Session 4 in student-starter/README.md: DockerFe/Dockerfile (TODO 1-13) and .dockerignore.

Checkpoint (everyone must pass before moving on)

- [] The page shows **Backend status: UP** and can add messages.
- [] Each student can draw the request path and point out where name resolution happens.

Errors you'll see in this session

Student sees	Cause and fix
npm ci ... requires a package-lock.json	Only package.json was copied. Copy package-lock.json too.
Backend status: DOWN	BACKEND_URL was wrong at build time, or the backend container isn't named backend / isn't on the same network.
Page loads without styles / JS 404s	.next/static wasn't copied into ./next/static.
Cannot find module '/app/server.js'	.next/standalone was copied to the wrong place. It must go to ./.
Very slow "transferring context"	There's no .dockerignore, so node_modules is being uploaded.

Note: Answer key: DockerFe/Dockerfile, DockerFe/.dockerignore and DockerFe/next.config.ts.

Session 5: Docker Compose

Goal: students replace all manual commands with one docker-compose.yml that starts reliably every time.

Timing

Min	Activity
0-10	Write all the session-4 commands on the board. Ask: "Would you run this every morning? Explain it to a new teammate?"
10-20	The structure of a compose file: services, networks, volumes. \${VAR:-default} and .env.
20-55	Students follow STEPS 1-3 in docker-compose.yml and test after each service.
55-70	The startup race (demo below) , then healthchecks plus condition: service_healthy (STEP 4).
70-85	Volumes: down / up keeps data, down -v wipes it. Run docker compose exec into each service.
85-90	Explain the final assignment (section 6). Hand out the setup guide.

Talking points

- Compose service names **are** the hostnames: db, backend. Same DNS idea as session 2, now automatic.
- depends_on alone only waits for the container to **start**, not for the app inside to be **ready**.
- A healthcheck is a command Docker runs repeatedly. service_healthy waits for it to pass.
- Named volumes live outside containers, so data survives down. down -v deletes it. The MySQL user and password are only created on the first start with an empty volume.

Live demo

```
# The startup race: plain depends_on, no healthcheck
docker compose up --build
#   backend-1 | ... Communications link failure
#   backend-1 exited with code 1           <- MySQL wasn't ready yet

# After adding healthchecks + condition: service_healthy
docker compose down -v && docker compose up --build -d
docker compose ps                # db (healthy), backend (healthy), frontend Up

# Volumes
docker compose down && docker compose up -d      # messages still there
docker compose down -v && docker compose up -d    # empty again
```

Note: This was verified on this project: with plain `depends_on: [db]` the backend exits with code 1 ("Communications link failure") because MySQL needs about 15-20 s on first start.

Student task

STEPS 1-4 in `student-starter/docker-compose.yml` (TODO 1-19).

Checkpoint (everyone must pass before moving on)

[] `docker compose down -v && docker compose up --build` starts everything without crashes.

[] Each student can explain what `down` and `down -v` each do to the data.

Errors you'll see in this session

Student sees	Cause and fix
<code>services.backend.depends_on ...</code> must be a list / mapping	YAML indentation. Use 2 spaces, and never tabs.
Access denied for user 'app'	The <code>.env</code> values changed after the volume was created. Run <code>docker compose down -v</code> .
Backend exits with code 1 on startup	No healthcheck or condition, so the backend starts before MySQL is ready (STEP 4).
dependency failed to start: container ... is unhealthy	The healthcheck command is wrong. Check it with <code>docker inspect --format '{{json .State.Health}}' <container></code> .
Changes don't appear	They ran up without <code>--build</code> .

Note: Answer key: `docker-compose.yml` and `.env.example` in the root folder.

3. How to debug with students

Teach the **same three steps** every time something breaks, and make students say them out loud:

1. `docker compose ps` Is it running? Is it healthy? Which ports?
2. `docker compose logs <svc>` What did the app itself say? Read the FIRST error.
3. `docker compose exec <svc> sh` Look from INSIDE: env, curl other services, ls files

Don't fix student errors for them. Ask "what do the logs say?" first. Reading logs is the most valuable skill they'll take from this course.

4. Final assignment and grading

Task: add **phpMyAdmin** as a 4th service (image `phpmyadmin`, reachable at `http://localhost:8081`, able to log in to db). Students hand in their `docker-compose.yml`.

How to grade: on a clean machine, run `docker compose down -v && docker compose up --build`.

Criterion	Points
The whole stack starts with one command, without manual steps	20
Backend and frontend Dockerfiles are multi-stage and run as non-root	15
Layer order allows caching (pom.xml / package files copied before source)	10
.dockerignore present and sensible in both services	5
db has a volume and a healthcheck; depends_on uses service_healthy	15
Services use service names (no localhost between containers)	10
phpMyAdmin works: on the network, port 8081, logs in using host db	15
No secrets hard-coded: uses .env / \${VAR}	5
Can explain one design choice in a 2-minute oral check	5
Total	100

Reference solution for phpMyAdmin

```
phpmyadmin:
  image: phpmyadmin
  environment:
    PMA_HOST: db                # the service name, not localhost
  ports:
    - "8081:80"                # phpMyAdmin listens on 80 inside the container
  depends_on:
    db:
      condition: service_healthy
  networks:
    - app-network
```

Note: A common mistake is "8081:8081": phpMyAdmin listens on port **80** inside the container. This is a good test of whether they really understand HOST:CONTAINER.

5. Quiz questions (with answers)

#	Question	Answer
1	What is the difference between an image and a container?	An image is a read-only template. A container is a running instance of it, with its own writable layer.
2	You delete a MySQL container. What happens to its data, and how do you prevent that?	It's lost, unless the data folder is on a volume (<code>-v db-data:/var/lib/mysql</code>).
3	What does <code>-p 4000:3000</code> mean?	Your computer's port 4000 forwards to port 3000 inside the container.
4	Does <code>EXPOSE 8080</code> make the app reachable from your browser?	No. It's documentation. Only <code>-p / ports:</code> publishes a port.
5	Why copy <code>pom.xml</code> before <code>src/</code> in the Dockerfile?	So the dependency-download layer stays cached when only the source code changes.
6	What ends up in the final image of a multi-stage build?	Only the last stage, plus whatever was explicitly copied into it with <code>COPY --from</code> .
7	Inside the backend container, what does <code>localhost</code> refer to?	The backend container itself. To reach MySQL, use the service name <code>db</code> .
8	Why can't the browser call <code>http://backend:8080</code> directly?	The browser runs outside the Docker network, where the name <code>backend</code> doesn't exist.
9	What is the difference between <code>depends_on: [db]</code> and <code>condition: service_healthy</code> ?	The first waits only for the container to start. The second waits until its healthcheck passes.
10	You changed <code>DB_PASSWORD</code> in <code>.env</code> and now the backend gets "Access denied". Why, and how do you fix it?	MySQL only creates users on the first start with an empty volume. Run <code>docker compose down -v</code> (this deletes the data).

6. Answer key: where each TODO is solved

Student file (student-starter/)	Solution (root folder)	Session
DockerBackend/Dockerfile Part A	Answer key box, Session 2 of this guide	2
DockerBackend/Dockerfile Part B	DockerBackend/Dockerfile	3
DockerBackend/.dockerignore	DockerBackend/.dockerignore	3
DockerFe/Dockerfile	DockerFe/Dockerfile	4
DockerFe/.dockerignore	DockerFe/.dockerignore	4
docker-compose.yml STEPS 1-4	docker-compose.yml	5
Final assignment	Section 4 of this guide	after 5

Note: The app code in `student-starter/` is identical to the answer key. With the solution files copied in, the whole stack was built and tested end to end, so any remaining student problem is in their Docker files, not in the app.